

WAMO: Toward Secure Browser Inference via Web Model Obfuscation in WebAssembly

Yitong Wang
Nanjing University of Aeronautics
and Astronautics
NanJing, China
tt.wang@nuaa.edu.cn

Pengfei Yu
Nanjing University of Aeronautics
and Astronautics
NanJing, China
nuaaypf@nuaa.edu.cn

Hao Han*
Nanjing University of Aeronautics
and Astronautics
NanJing, China
hhan@nuaa.edu.cn

Jingjing Gu
Nanjing University of Aeronautics
and Astronautics
NanJing, China
gujingjing@nuaa.edu.cn

Fengyuan Xu
Nanjing University
NanJing, China
fengyuan.xu@nju.edu.cn

Xinyi Huang
Nanjing University of Aeronautics
and Astronautics
NanJing, China
huangxinyi@nuaa.edu.cn

Abstract

Artificial intelligence (AI) models are increasingly deployed directly in web browsers to enable low-latency, privacy-preserving inference. While this shift offers significant usability and scalability benefits, it also exposes model code and parameters to untrusted environments, leaving them vulnerable to theft, reverse engineering, and tampering. Our analysis demonstrates that existing JavaScript-based inference frameworks are highly susceptible to model extraction, posing serious security and intellectual property risks. To address this gap, we present WAMO, a WebAssembly-based obfuscation framework that secures browser-side AI models. WAMO introduces a comprehensive conversion pipeline that translates mainstream model formats into Wasm-native modules, applying model-specific obfuscation at the Wasm layer to target weights, operators, and computation graphs. This design shifts model execution from easily inspected JavaScript assets to hardened Wasm binaries, significantly raising the difficulty of static and dynamic analysis. Evaluation shows that WAMO increases cyclomatic complexity by 71.0% and Halstead effort by 455.57%, while incurring <1% accuracy loss and no inference slowdown.

Keywords

Model Protection, Model Theft, WebAssembly, Binary Obfuscation

ACM Reference Format:

Yitong Wang, Pengfei Yu, Hao Han, Jingjing Gu, Fengyuan Xu, and Xinyi Huang. 2026. WAMO: Toward Secure Browser Inference via Web Model Obfuscation in WebAssembly. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792297>

*Corresponding author

1 Introduction

The rapid integration of artificial intelligence (AI) into web applications has transformed the way users interact with online services [49]. Traditionally, AI inference has been performed on powerful backend servers, with web browsers serving as thin clients that are primarily responsible for user interface rendering. However, recent advances in model optimization, client-side hardware acceleration, and browser-based execution frameworks such as TensorFlow.js, Brain.js, and ONNX runtime Web [27] have enabled the deployment of AI models directly within the browser.

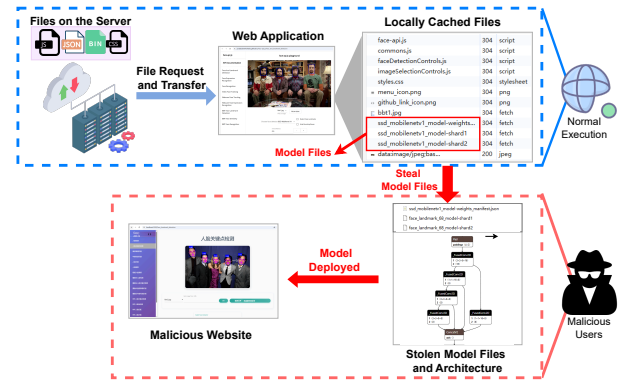


Figure 1: The process of model loading on the browser side and the process of malicious users stealing the model.

Running AI models in browsers offers several practical advantages [41]. First, it enables personalized and responsive user experiences by processing data locally [23], thereby mitigating network delays and server bottlenecks. Second, client-side inference reduces the need to transmit sensitive data over networks, aligning with privacy and compliance requirements [2]. Third, distributing computational workloads to end devices enables service providers to scale more efficiently, particularly in scenarios involving millions of concurrent users [15]. These benefits are driving adoption across diverse domains, including real-time translation, speech recognition, image enhancement, and interactive recommendation systems.



This work is licensed under a Creative Commons Attribution 4.0 International License. [WWW '26, Dubai, United Arab Emirates](https://creativecommons.org/licenses/by/4.0/)

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792297>

Despite these opportunities, deploying AI models in browsers poses critical security threats. Once a model is delivered to the client, it becomes accessible to potentially untrusted environments, exposing it to threats such as model theft, reverse engineering, adversarial manipulation, and integrity compromise [46]. Unlike server-side deployments, where the model remains under the provider's control, client-side deployments must assume that adversaries may inspect or extract sensitive intellectual property from the model. Figure 1 presents the process by which malicious users steal models. If attackers can monitor client execution and network interactions and analyze model architecture, they can steal JavaScript front-end models and fully reproduce them on the client side.

Browser-side model deployment follows two modes. The first involves native JavaScript frameworks (e.g., TensorFlow.js [43]), where the model and its execution logic are managed as libraries within the JavaScript virtual machine, often delegating intensive operations to a WebAssembly (Wasm) backend. The second mode uses external model files (e.g., ONNX, TFJS lite [35]), which are dynamically loaded by browser APIs and executed via hardware acceleration (WebGL/WebGPU) or precompiled Wasm modules. In both approaches, Wasm is commonly employed to achieve near-native inference speeds. However, prior work has predominantly focused on leveraging Wasm for these performance benefits, while its security capabilities have been largely overlooked. In fact, Wasm offers sandboxing, boundary checks, and minimized host interaction [6], making it a promising candidate for building a security boundary. However, realizing this vision presents several key challenges that must be addressed.

C1: Lack of a complete conversion path from existing frontend inference frameworks to Wasm-native models. While existing frameworks can compile source code from languages like C++ or Rust into Wasm [33, 45, 56], no direct toolchain exists for converting standard AI model formats. Trained models are typically stored as text or binary files (e.g., .bin, .json, .pth, or .onnx), which are designed to be interpreted, not compiled. To complete the conversion, these model files must first be transformed into a compilable representation. Furthermore, general C++ operator libraries often fail to compile for Wasm due to unmapped semantics, such as exception handling, platform-specific intrinsics, and SIMD/vectorization differences. This mismatch results in undefined symbols or runtime errors. Although limited attempts like TVM [48] provide partial support for some operators, they are insufficient for full inference. Consequently, the ecosystem lacks both toolchains for converting mainstream models into native Wasm and Wasm-friendly operator libraries.

C2: Lack of efficient, model-targeted obfuscation at the Wasm layer. Obfuscation for browser-side models can be performed at several stages—C++, LLVM IR, or directly at the Wasm layer. Existing work mainly targets C++ [20] or LLVM IR [7]. The former often introduces exponential code bloat in complex inference engines and yields protections weakened or lost after compilation to Wasm. The latter, while more general, operates at too low an abstraction level to capture model-specific semantics, limiting its effectiveness for AI workloads. Obfuscation directly at the Wasm layer is more attractive: it is close to execution, incurs less semantic loss, and aligns with the runtime environment. However, Wasm's design makes this approach particularly difficult. Its stack-based

execution model eliminates much of the high-level structure of the original program, producing long, verbose instruction sequences with little semantic richness. This lack of abstraction obscures the boundaries between operators, weights, and computation graphs, making it hard to apply fine-grained, model-aware transformations. Current Wasm obfuscation studies remain confined to generic control-flow transformations or bulk data encryption [8], which fail to protect the unique components of neural network inference. As a result, there is still no practical approach for efficient, model-targeted obfuscation at the Wasm layer.

To tackle these challenges, we present WAMO, a framework that obfuscates browser-deployed AI models through a novel two-stage process. Our key insight is to shift the protection strategy from interpreting vulnerable JavaScript assets to executing a pre-compiled, hardened Wasm binary. WAMO comprises two components: a model conversion module and a security enhancement module. The conversion module implements a complete pipeline that transforms mainstream model files into executable, native Wasm modules. By leveraging a robust set of model parsers and custom Wasm-friendly C++ operator libraries, it reliably converts model files into Wasm representations. The security enhancement module introduces model-aware obfuscation techniques designed specifically for the Wasm layer. Unlike generic code transformation, it applies multi-dimensional obfuscation within the Wasm code that targets the unique semantics of an AI model, including weights, operators, and computation graphs. Our evaluation across 14 models shows that WAMO provides robust security enhancements against both static and dynamic analysis.

To summarize, the contribution of this work is as follows:

- To the best of our knowledge, we provide the first systematic investigation of model stealing risks in web-deployed AI models. We design a generic stealing method and implement a verification tool to evaluate the susceptibility of JavaScript-based inference frameworks, revealing that models (including commercial ones) delivered to the browser can be efficiently extracted and reverse-engineered by adversaries.
- We introduce the first WebAssembly-based obfuscation techniques tailored to web-deployed AI models. These techniques go beyond traditional WebAssembly obfuscation by targeting the unique semantics of AI inference models, thereby significantly increasing the difficulty of model extraction and reverse engineering in the browser.
- We develop WAMO, a model conversion and obfuscation tool that implements over 140 WebAssembly-optimized operators and provides an automated pipeline from mainstream model formats (e.g., TensorFlow.js, ONNX) to obfuscated Wasm-native inference. We conduct extensive experiments on 14 representative models, demonstrating that our approach achieves strong resistance against model extraction while maintaining execution efficiency and compatibility. WAMO is open source at <https://github.com/Eudora-tt/WAMO>.

2 Background and Motivation

The widespread adoption of frontend inference frameworks like TensorFlow.js enables powerful applications but exposes valuable intellectual property to untrusted clients. This section first analyzes

the attack surface of web-deployed models, then provides empirical evidence of their vulnerability through a practical extraction, thereby establishing the critical need for a new security framework.

2.1 Attack Surface Analysis

The primary vulnerability of frontend inference frameworks stems from how models are loaded and stored. Models are typically fetched via asynchronous requests as a set of files (e.g., `.json`, `.bin`, `.onnx`). Although HTTPS secures data in transit, it offers no protection against client-side attack vectors, such as man-in-the-middle attacks, untrusted proxies, or compromised clients [13]. Once downloaded, these model files are cached locally using memory, HTTP disk cache, Cache Storage, or IndexedDB [52] to enhance access efficiency. These caching mechanisms create client-accessible copies. Attackers can extract these files using developer tools, local file systems, or malicious extensions, presenting significant risks of intellectual property leakage and misuse.

Furthermore, the model’s execution logic is also exposed. Frameworks deliver the model to a backend for computation, typically WebGL, JavaScript, or Wasm [5, 34, 44, 53]. WebGL maps operators to GPU shaders for parallel computation, while JavaScript backends interpret operators directly. Even Wasm, while offering a sandboxed and efficient execution environment for compiled C/C++, Rust, or Go code [14, 33, 45, 56], clients can still inspect the module itself and the scheduling logic. The transparency of the entire pipeline from storage to execution constitutes a significant attack surface that remains largely unaddressed.

2.2 Real-World Model Extraction

To empirically validate these threats, we developed a lightweight model stealing tool for frontend models. In TensorFlow.js, weight (`.bin`) and architecture (`.json`) files are transmitted to the client and cached in the browser, enabling the tool to capture them and reconstruct the trained model using TensorFlowjs_converter. Models deployed with ONNX Runtime Web can be extracted similarly through stolen `.onnx` files. For Brain.js, where models are stored as JavaScript objects, the tool parses extracted JavaScript files to reconstruct JSON representations and recover complete model information. After identifying input and output structures, the stolen models can be directly reused in browsers with full functionality.

We examined seven websites using web-deployed model inference, including commercial sites and smaller ones, as shown in Table 1. The websites use TensorFlow.js, Brain.js, and ONNX Runtime Web, with TensorFlow.js preferred by commercial sites and the others mainly for small-scale or demo purposes. We successfully extracted models from all seven websites, collected various model information, and redeployed them on our own test site.

These empirical studies demonstrate that JavaScript-based web-deployed models are highly vulnerable during loading and inference. Existing security paradigms struggle to address such risks. Traditional encryption [4, 17] provides limited protection, as decrypted models appear in browser caches. Differential privacy [29, 57] degrades performance and requires network-specific tuning. Homomorphic encryption [9, 16, 24] incurs prohibitive computational costs, making client-side execution impractical.

Table 1: Summary of Tested Models

Model Function	Model Framework	Operators	Commercial?
Makeup [31]	TensorFlow.js	252	Yes
Animation	TensorFlow.js	187	No
Online Meeting [25]	TensorFlow.js	977	Yes
Fitness [50]	TensorFlow.js	396	Yes
Color Readability	Brain.js	26	No
Cart-Pole	Brain.js	43	No
Audio Prediction	ONNX Runtime Web	96	No

3 System Overview of WAMO

As discussed, JavaScript-based model execution on the web poses significant security risks, enabling attackers to extract key information and reconstruct models. To mitigate this, we propose WAMO, a Wasm-based protection system that ensures secure and reliable web deployment. The WAMO is divided into two modules: model conversion module and security enhancement module, as illustrated in Figure 2. The model conversion module transforms models from arbitrary formats into Wasm model files. The security enhancement module provides multi-faceted obfuscation for Wasm model files to strengthen browser runtime security.

The system workflow comprises four stages—model loading, conversion and compilation, security enhancement, and browser deployment—forming an end-to-end automated pipeline. Users provide trained models, such as Tensorflow, PyTorch, or ONNX files, which are automatically recognized and converted into a standardized intermediate representation via a unified parser, then transformed into semantically equivalent C++ code. The system compiles the C++ code into Wasm for browser execution. In the security stage, users can select obfuscation methods. Finally, the model is deployed to the browser for secure local inference, minimizing backend dependency and transmission leakage. This automated workflow is user-friendly and suitable for web applications requiring stringent model protection. The design principles comply with secure and portable web model deployment standards (see Appendix B).

4 Model Conversion Module

This section describes the model conversion module, including its design and implementation. It loads models, extracts architectures, stores weights, and invokes necessary operators to convert them into Wasm model files, without relying on external libraries.

4.1 Our Operator Library

Studies [22, 55] show that dependence on external machine learning operator libraries can introduce buffer overflows, memory leaks, and privilege bypass. Malicious inputs may trigger these vulnerabilities, resulting in denial-of-service, data leakage, or remote code execution. Such libraries are complex and tightly coupled with low-level resources, rendering models "black boxes" that are hard to audit, and expanding the attack surface and dynamic analysis can be circumvented, enabling exploitation of known vulnerabilities [28].

To mitigate security risks, we developed a custom C++ operator library, offering functions executed exclusively within the Wasm

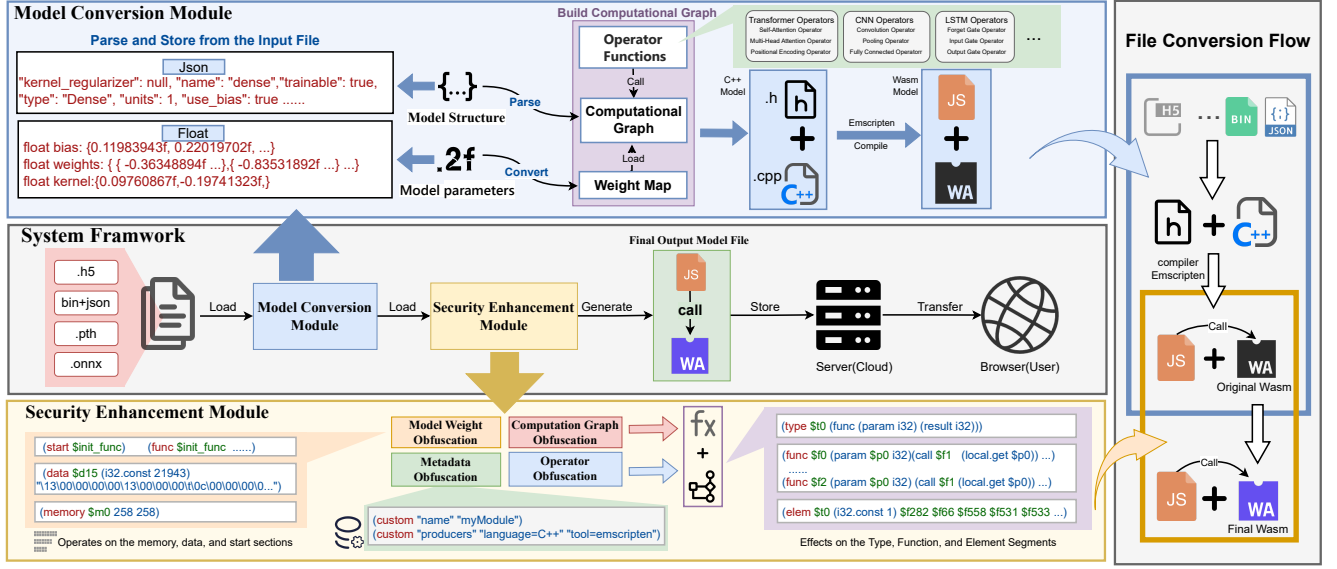


Figure 2: The complete framework and some details of WAMO system.

sandbox, without relying on external libraries. This library is Wasm-compatible and efficient, overcoming compatibility problems of existing C++ operator libraries in Wasm compilation or runtime.

Our custom operator library includes more than 140 operators, such as SVM, random forest, RNN, CNN, LSTM, GAN, and Transformer. It supports standard model conversion and is fully Wasm-compatible, ensuring smooth execution in web environments. Details of the operator library are provided in Appendix C.

Operators are implemented as C++ functions with well-defined inputs and outputs, where parameters like weights and biases serve as function inputs, and computation is performed internally for modular reuse. For example, an attention operator takes a sequence of vectors and learned weight matrices as input and produces a weighted context vector. Inputs are projected into query (Q), key (K), and value (V) matrices, and the dot product of Q and K is scaled, passed through softmax to obtain attention weights, and then multiplied with V to generate the output.

4.2 Model Parser and Converter

The model parser extracts configuration and parameter information from various model files and stores them as C++ header files for use without a complete deep learning framework. It supports multiple formats, using the appropriate Python parser for each file, and traverses layers to capture type, input shape, weights, and settings. Weights are converted to multi-dimensional floating-point arrays, and layer details are serialized to JSON, generating string constants for the model configuration. This allows direct access to weights and configurations in C++ without relying on external libraries.

The model converter integrates architecture and weight parameters to perform forward computations via operators, generating a complete computation graph. The graph is layered, with operators linking inputs and weights. Stored architectures initialize the network, composed of layers like Conv2DLayer and DenseLayer, each

containing type, weights, and biases. The Network class stores layers as a vector of smart pointers (`shared_ptr<BaseLayer>`). Layers inherit from `BaseLayer`, using polymorphism to define distinct members and behaviors, providing the computation framework.

Once the network is initialized, the system iterates through each layer, applying the appropriate operator according to its type. Operators fetch weights, biases, strides, padding, and inputs, perform computations, and forward results, ensuring correct data flow between layers. This process constructs a complete computation graph for inference output.

These steps produce a model with the complete computation process. Operators reside in C++ headers, and computation graph construction code in C++ sources. Using the Emscripten compiler, they are compiled into a Wasm model file covering the entire inference process. The Wasm model lacks input handling; the browser must supply data via JavaScript, which calls the Wasm model file, ensuring full consistency with the original computation graph.

5 Security Enhancement Module

Wasm models produced by the converter remain vulnerable against security threats. Using static analysis tools like `wasm2wat` [51], attackers can translate these files into readable text and extract critical model information from data section and control flow. To mitigate this risk, we introduce security enhancements. Figure 3 illustrates the obfuscator developed to strengthen Wasm security.

5.1 Data Obfuscation

In Wasm model files containing models, weights reside in the data section and may be extracted via `wasm2wat` [51]. To enhance protection, we encrypt weights at deployment and combine this with fake decryption obfuscation and dummy weight injection for layered defense, as illustrated in Figure 4.

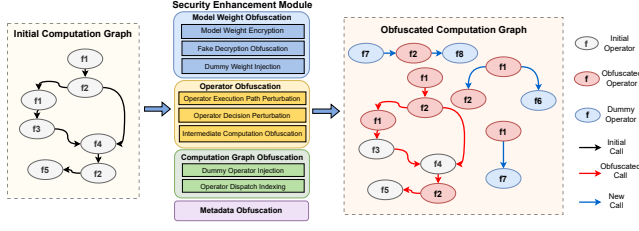


Figure 3: Security enhancement module. The figure illustrates the components of the module and the obfuscation effects on the computation graph.

5.1.1 Model Weights Encryption. Each weight’s IEEE754 mantissa is encrypted through multiple Feistel rounds, with per-weight perturbations from randomly generated mantissa keys. The keys are RSA-encrypted and embedded in the header to avoid leakage. Encrypted weights are reconverted into floating-point form, stored in the data section, and appear in binary within the wat file to resist reverse engineering. At runtime, on-demand decryption restores keys for inference, decrypts weights, and destroys them when idle, ensuring sensitive data exists only when required.

5.1.2 Fake Decryption Obfuscation. In the Wasm module’s start section, we embed a fake "decryption" flow to simulate weight decryption at load time. Because start executes automatically on instantiation, analysts are misled to believe weights were decrypted with our mock keys during loading, although the routine does not affect actual execution or weight access.

Start points to a void $\rightarrow$ void function in the system code section where fake decryption logic is injected. The logic uses `i32.const` and `i32.store` to write mock keys to high-offset memory, disguises key checks with arithmetic and conditional jumps, and reconfigures the memory segment so all fake writes redirect to a new tail region (e.g., `memory[0x1FF00]`), preserving normal data flow.

5.1.3 Dummy Weight Injection. To effectively obscure genuine model parameters, we insert substantial fake weight data into the data section, while real weights are securely encrypted and hidden. The fake segments are crafted to mimic ciphertext, embedding randomly generated floating-point values simulating the current model and weight fragments sampled from diverse real models, distributed across multiple offsets. To avoid conflicts with authentic weights, these entries are organized using a complex `i32.const` + `end` pattern. Certain fake segments even replicate realistic indexing methods and memory layouts, thereby increasing the likelihood that adversaries will misinterpret them as valid parameters.

5.2 Operator Obfuscation

Operator-level obfuscation conceals the internal computations of each operator. Modifying data storage routes and execution order, it obscures an attacker’s ability to infer operator semantics. We realize three techniques: operator execution path perturbation, operator decision perturbation, and intermediate computation obfuscation.

5.2.1 Operator Execution Path Perturbation. We flatten and reconstruct function execution paths into complex operator-level scheduling structures to increase reverse-engineering difficulty. Each

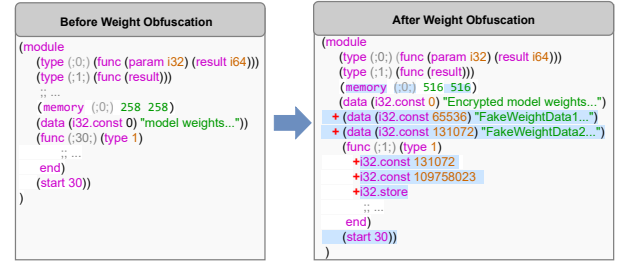


Figure 4: Model weight obfuscation: weight encryption, fake decryption obfuscation, and dummy weight injection.

Wasm operator function is split into basic blocks via linear scanning with stack-depth tracking. Branch and jump instructions (e.g., `br`, `br_if`, `return`, `if`, `else`, `end`) mark block boundaries, with offsets and stack deltas recorded to preserve semantics.

The basic blocks are reordered and encapsulated within a unified operator scheduling loop. Using a state variable (e.g., `i32.state`) as the path index, the scheduler assigns identifiers to blocks and drives execution through `br_table` jumps with state updates, thereby flattening multi-entry/multi-exit control flow into a single-entry dispatcher. This transformation obscures the actual execution path, rendering operator sequences, branch successors, and data flows highly non-intuitive. The detailed algorithm and its implementation results are presented in Appendix D.

5.2.2 Operator Decision Perturbation. This obfuscation targets critical conditional branches within operators, focusing on decision points that govern execution flow. While maintaining semantic equivalence, the conditional logic and path sequence are altered. For instance, a branch "execute path A if the condition holds, otherwise path B" may be transformed into "execute path A if the condition does not hold, otherwise path B," with the paths’ internal order swapped. This effectively scrambles the operator’s execution sequence, significantly hindering attackers from reconstructing its true logical structure [30].

5.2.3 Intermediate Computation Obfuscation. This obfuscation disrupts data flow by perturbing intermediate stack values within operators. Redundant `i32.const` and `drop` instructions modify the stack state of intermediate results, while `swap` and `dup` reorder or replicate top-stack elements, altering operator behavior. Additional dead computation paths, including superficially meaningful but result-neutral sequences (e.g., consecutive `i32.add` and `i32.sub`), increase analysis complexity. Identity operations like `i32.add 0`, `i32.mul 1` further impede stack tracking. These low-overhead techniques effectively obscure internal data dependencies and intermediate computation order.

5.3 Computation Graph Obfuscation

Computation graph obfuscation means obscuring operator call relations and invocation order at the computation-graph level, making it difficult for attackers to reconstruct the full operator-call graph. To achieve this, we design dummy operator injection and operator dispatch indexing to increase reconstruction difficulty from multiple dimensions.

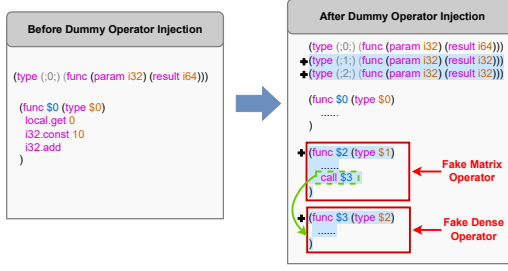


Figure 5: Impact of dummy operator injection on sections.

5.3.1 Dummy Operator Injection. We obfuscate the operator-call network by inserting forged operator functions to hinder identification of genuine operators. Fake function signatures are randomly added to the type section; parameter and return types are chosen from `i32`, `i64`, `f32`, and `f64` to form `WasmFunctionType` entries.

Function bodies are generated for each fake signature and placed in the `func` section. Some forged operators derive from compiled outputs of our operator library. We inject benign model operators that do not alter the original computation graph; they may call one another or existing operators to form spurious dependency chains. Selected functions embed no-op patterns and elaborate control flow [10], including unused locals, side-effect-free constants, and arithmetic such as `local.get`, `i32.add`, combined with `block`, `loop`, and `br` to produce syntactically complex but semantically inert paths. The final effect is shown in Figure 5.

5.3.2 Operator Dispatch Indexing. We employ a dispatch table to redirect selected operator calls from static call to index-driven `call_indirect`, obscuring inter-operator call relations [11] and complicating static reconstruction of the computation graph. In practice, all call instructions are identified, their target function indices extracted, and added to the `elem` table, defining the set of functions accessible by index. Each original call is replaced with an `i32.const` plus `call_indirect` sequence. The specific implementation results are presented in Appendix D.

5.4 Metadata Obfuscation

Beyond model-level obfuscation, Wasm supports special custom sections, which we exploit for metadata obfuscation. This involves inserting fake custom sections to introduce irrelevant information, misleading static analysis without affecting runtime semantics. For instance, attackers cannot infer the compiler or accurately determine the source language of the Wasm model file. Since these sections are ignored during execution, they naturally serve as obfuscation carriers.

```
(module
  +(@custom "producers" "2 entries: language=Rust, processed-by=rustc 1.70.0")
  +(@custom "name" "module=wasmmemod, funcs=[_funcA, _funcB]")
  +(@custom "target_features" "features=[atomics, simd]")
  +(@custom "custom-data" "{\\"compiler\\": \\"rustc\\", \\"version\\": \\"1.70.0\\"}")
)
```

Figure 6: Metadata obfuscation diagram.

We add multiple fake custom sections to generated Wasm files to embed fabricated metadata (Figure 6). These include simulated compiler info (e.g., `pretend_clang`, `fake_wasi_sdk`), name sections for function/local names, `target_features`, `sourceMappingURL` with fake debug paths, and custom metadata. For specific compilers (e.g., `clang`, `rustc`), we insert fake linking sections with random symbol-table entries. These sections are embedded in the module binary and ignored at runtime but can mislead static analysis.

6 Evaluation

To provide a comprehensive evaluation of WAMO, we aim to address the following research questions:

RQ1: Can the security enhancement module of WAMO significantly improve the resistance of model files to static analysis?

RQ2: Can the WAMO system significantly enhance the resistance of model files to dynamic analysis?

RQ3: To what extent does the WAMO system affect the runtime performance and accuracy of the models?

RQ4: What advantages does our approach offer compared to existing obfuscators?

6.1 Experimental Setup

To address the above research questions, we collected 14 original TensorFlow.js models. The model file sizes range from 26 KB to 56 MB. For each TensorFlow.js model (Baseline), we produced two corresponding Wasm variants. First, the model conversion module generated a converted but unobfuscated Wasm file, which we denote as *Wasm-Base*. Next, the security enhancement module applied encryption and obfuscation techniques to this file, resulting in the obfuscated version, referred to as *Wasm-Obf*. Detailed experimental environment settings are shown in Appendix E.

To address RQ1, we measured cyclomatic and Halstead complexity [3, 19, 21, 21, 32]. Baseline TensorFlow.js models were excluded because they are directly readable via browser developer tools and offer negligible resistance to static analysis. For RQ2, we measured normalized memory-distribution entropy during inference. Relevant formulas and metric definitions are provided in Appendix F. To answer RQ3, we measured model size, loading time, accuracy, and indicators to evaluate system usability and effectiveness of WAMO.

6.2 RQ1: Static Analysis

Attackers reverse-engineering Wasm modules often rely on static symbolic executors for semantic analysis and path exploration. Analysis time is thus a key metric for evaluating control-flow complexity and static analysis difficulty. To test the analyzability of generated models, we used unobfuscated Wasm models from the WAMO conversion module and applied tools such as Manticore [37] and SeeWasm [40]. The tools frequently report errors like "unsupported semantics, only basic instructions supported" and fail to handle non-MVP features in Emscripten-generated Wasm model files, including multi-value returns, exception handling, memory64 extensions, SIMD instructions, and complex control flow. Some also encounter "unknown opcode" errors on custom sections or high-version opcodes, preventing symbolic execution. These results demonstrate that our Wasm model files, due to their semantic

and structural complexity, effectively resist mainstream symbolic executors, with most tools unable to complete initialization.

From a manual static-analysis viewpoint, attackers typically convert Wasm to wat for inspection. Prior to weight obfuscation, weights reside plainly in the data section and are easily extracted. After our security enhancements, weights are interleaved with fabricated binary data, obscuring true data segments. An initialization decryptor at program entry inserts fake decryption routines and numerous key-like constants, complicating key discovery. Emcc compilation in a special mode emits numeric function names to hinder function-level analysis. Metadata obfuscation additionally injects randomized compiler and source language metadata to confuse static analysis.

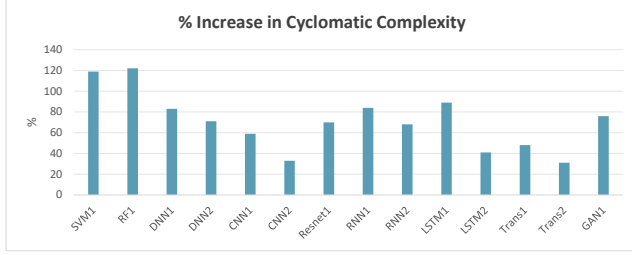


Figure 7: Cyclomatic complexity increase rate after obfuscation.

For quantitative evaluation, we measured cyclomatic complexity of Wasm model files before and after obfuscation. As shown in Figure 7, our security module increased complexity by an average of 71.0%, with improvements ranging from 31% to 122%.

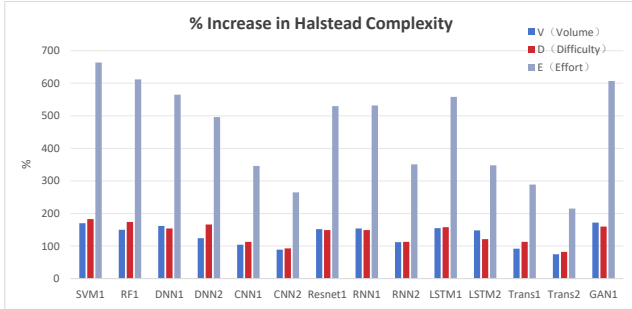


Figure 8: Halstead Complexity increase rates in Volume, Difficulty, and Effort after obfuscation.

We evaluated Halstead complexity metrics on the converted wat files. The figure 8 presents the growth rates of Volume, Difficulty, and Effort after obfuscation. Respectively, these metrics represent program information content, comprehension difficulty, and overall cognitive effort. Our obfuscation raised them by 132.79%, 137.71%, and 455.57%, with all 14 models achieving notable improvements.

6.3 RQ2: Dynamic Analysis

This section compares the three groups in terms of resistance to dynamic analysis. Our experiments indicate that current dynamic-analysis tools cannot directly instrument our Wasm model files

automatically, requiring attackers to resort to manual analysis. Typical manual workflows include observing the page load to locate model initialization and key execution entry points; intercepting or perturbing inference to trace function calls, jumps, and memory changes; inspecting intermediate computations, I/O tensors, and candidate parameters to infer types and roles; partially recovering weights and structure to reveal core inference logic; and ultimately extracting critical data to reconstruct or exploit the model locally.

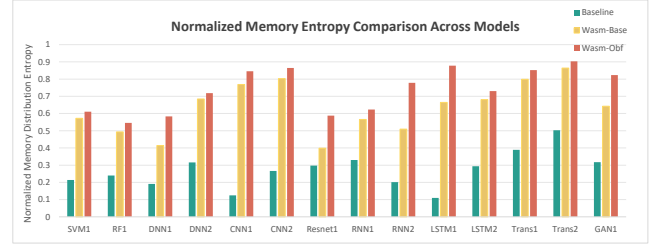


Figure 9: Comparison of normalized memory distribution entropy across three model groups.

We evaluated the normalized memory distribution entropy of 42 model files during inference (Figure. 9). The *Wasm-Base* group produced by the model converter exhibited a 164.73% average improvement over the Baseline group. The *Wasm-Obf* group achieved an additional 18.85% increase compared with non-obfuscated files. Relative to the original TensorFlow.js files, the obfuscated models yielded a 215.63% improvement.

In our dynamic analysis, we compared the inference paths of three models through debugging. The Baseline model, when invoking `model.predict()`, reveals a clear semantic call chain with complete names and explicit relations, allowing attackers to observe intermediate tensors and identify model structures and partial weights, showing high observability. In contrast, the *Wasm-Base* model employs numeric indices, with many more functions and complex call paths, but its inference remains centralized and linear, making activations and weights still traceable via memory changes. The *Wasm-Obf* model introduces control-flow obfuscation through a unified entry and multiple jumps. Although its stack structure resembles that of *Wasm-Base*, scrambled execution, absent semantic cues, and obscured breakpoints markedly diminish observability, concealing both inference logic and memory patterns.

6.4 RQ3: Performance and Accuracy Comparison

The system provides developers with secure, reliable models for browser-based inference using local resources. Since model size and execution speed are critical, we assessed file size, loading time, inference time, and memory usage, comparing accuracy before and after conversion and obfuscation. Results demonstrate that the system enhances security while maintaining minimal performance overhead, particularly excelling in inference efficiency.

Regarding file size, conversion from Baseline to *Wasm-Base* increases file size to 1.49 times the original, and further obfuscation to *Wasm-Obf* raises it to 1.66 times. The growth is within expected overhead. In memory usage, obfuscated models incur an average

Table 2: File size (kb) and memory usage (mb) across different models and implementation formats.

	SVM1	RF1	DNN1	DNN2	CNN1	CNN2	Resnet1	RNN1	RNN2	LSTM1	LSTM2	Transf1	Transf2	GAN1
File Size (kb)														
Baseline	26	31	350	1648	7579	14286	608	382	1498	469	5283	6674	55468	1347
Wasm-Base	48	59	587	2382	10815	18793	1071	571	2144	687	7241	9750	63682	2048
Wasm-Obf	99	125	1048	3964	17596	25437	1529	995	3898	1209	11993	13534	89906	3571
Memory Usage (mb)														
Baseline	8.1	9.3	13.2	26.4	28	71	20.7	15.4	16	15.2	30.3	38.1	92.6	17.4
Wasm-Base	9.4	10.8	16.8	32.1	40.1	15.4	27.2	18.2	17.3	16.3	42.5	51.2	114.8	21.2
Wasm-Obf	10.8	12.5	17.2	36.8	42	18.2	28.8	21.6	18.5	16.8	48.9	58.5	126.2	21.9

Table 3: Model loading time and inference time (in milliseconds) across different models and implementation formats.

	SVM1	RF1	DNN1	DNN2	CNN1	CNN2	Resnet1	RNN1	RNN2	LSTM1	LSTM2	Transf1	Transf2	GAN1
Model Loading Time (ms)														
Baseline	10.4	13	28.3	44.8	46.8	73.2	12.5	30.1	35.2	31.7	50.3	56.3	113.7	40.6
Wasm-Base	11.2	<u>13.9</u>	29.5	49	53.4	84.5	17.2	<u>32</u>	37.3	32.5	59	64	131.8	50.2
Wasm-Obf	12.5	15.3	<u>31.2</u>	58.3	68.1	<u>90.6</u>	19.5	33.8	<u>40</u>	<u>35.6</u>	63.4	79.3	147.5	<u>51.5</u>
Model Inference Time (ms)														
Baseline	0.7	1.1	4	11.2	93.2	148.5	18.1	3.4	5.7	6.3	21.5	122.3	321.4	11.4
Wasm-Base	<u>0.31</u>	0.6	<u>1</u>	6.5	68.9	103.7	7.2	<u>1.8</u>	2.5	<u>2.2</u>	20.3	86.5	242.7	<u>5.2</u>
Wasm-Obf	0.35	<u>0.62</u>	1.4	7.3	74.5	114.6	<u>7.6</u>	1.9	<u>2.9</u>	2.6	24.2	96.8	268.5	6.5

increase of 28.93%, with a maximum under 40%. For certain models, like LSTM1, the difference is merely 1–6MB, demonstrating effective memory management. File size and memory usage of the three model groups are shown in Table 2.

For loading time, all models exhibit increased delays after obfuscation, but the growth remains within a reasonable range (10%–30%), which is unlikely to noticeably impact users in practice. In inference time, the system demonstrates significant advantages. *Wasm-Base* achieves over 40% average speedup compared with the Baseline. Even after obfuscation, *Wasm-Obf* outperforms the Baseline; for instance, CNN2 is roughly 10% slower than *Wasm-Base* but still 22% faster overall. The specific loading time and inference are shown in the Table 3, with some relatively better results underlined.

Accuracy was evaluated by testing each model five times across five datasets, comparing results before and after conversion. *Wasm-Base* differs from Baseline by at most 1%, primarily due to operator implementation details, execution environment, and Wasm type constraints. Importantly, obfuscation leaves accuracy unchanged, indicating model semantics and structure are preserved.

6.5 RQ4: Comparison with Existing Obfuscators

The compilation path of our Wasm model files goes from C++ through IR to WebAssembly, so we evaluated three mainstream compilers at successive stages, including Tigress [1] (C++ level), Wasm-mutate [7] (IR level), and WASMixer [8] (Wasm level). Table 4 compares these three obfuscators with our obfuscator. A check mark (✓) indicates ideal or fully effective performance, a cross (✗) indicates lack of support or very poor performance, and a circle (○) indicates partial support or occasional feasibility.

As the first model-targeted transformer and obfuscator, only our security enhancement module fully supports Wasm model

Table 4: Comparison of Obfuscators.

Feature	Tigress	Wasm-mutate	WASMixer	WAMO
Advanced Semantics	✓	✗	✗	✓
parameters Protection	✓	✗	○	✓
Compilation Time	✗	✓	✓	✓
Model Applicability	○	✗	✗	✓
Output Stability	○	✗	○	✓

files. Tigress [1] faces prohibitive compilation overhead from larger model sizes, whereas Wasm-mutate [7] and Wasmixer [8] cannot obfuscate advanced semantics such as SIMD in our models and lack model-level protections.

7 Conclusion

We presented a WebAssembly-based framework for protecting machine learning models deployed in browsers. Our approach converts diverse model formats into C++ with custom operators, compiles them into Wasm, and applies tailored obfuscation to strengthen resistance against static and dynamic analysis. Experiments show that the framework enhances model security while preserving accuracy and maintaining acceptable runtime and size overhead. This work underscores the need for binary-level protection in browser-side inference and points to future directions such as model watermarking, runtime integrity verification, and adaptive obfuscation.

Acknowledgments

This work was supported in part by NSFC under Grant 62425205, Grant 62272224; in part by Yangtze River Delta Science and Technology Innovation Community Joint Research Project under Grant 2024CSJZN00400.

References

- [1] [n. d.]. Tigress: A Diversifying Virtualizer/Obfuscator for C. <https://tigress.wtf>. Accessed: 2025-10-07.
- [2] Anton An. 2025. Client-Side Processing and Storage of Large Data Volumes in Web Applications Using AI Techniques. *Computational Intelligence and Machine Learning* 6, 1 (2025), Article A006. doi:10.36647/CIML/06.01.A006
- [3] P Chitti Babu, A Narasimha Prasad, and D Sudhakar. 2013. Software complexity metrics: a survey. *International Journal* 3, 8 (2013).
- [4] Ishita Bajaj and RK Aggarwal. 2019. RSA Secured Web Based Steganography Employing HTML Space Codes and Compression Technique. In *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*. IEEE, 865–868.
- [5] Brain.js Contributors. 2023. Brain.js: Neural networks in JavaScript. <https://github.com/BrainJS/brain.js>. Accessed: 2025-09-11.
- [6] Bytecode Alliance / Wasmtime. 2025. *Security — Wasmtime Documentation*. <https://docs.wasmtime.dev/security.html>. Accessed: 2025-09-11.
- [7] Javier Cabrera-Arteaga, Nicholas Fitzgerald, Martin Monperrus, and Benoit Baudry. 2024. Wasm-Mutate: Fast and effective binary diversification for WebAssembly. *Computers & Security* 139 (2024), 103731.
- [8] Shangdong Cao, Ningyu He, Yao Guo, and Haoyu Wang. 2024. WASMixer: Binary Obfuscation for WebAssembly. In *European Symposium on Research in Computer Security*. Springer, 88–109.
- [9] Edward Chou, Josh Beal, Daniel Levy, Serena Yeung, Albert Haque, and Li Fei-Fei. 2018. Faster cryptonets: Leveraging sparsity for real-world encrypted inference. *arXiv preprint arXiv:1811.09953* (2018).
- [10] Fred Cohen. 1993. Operating system protection through program evolution. *Computers & Security* 12, 6 (1993), 565–584.
- [11] Christian Collberg, Clark Thomborson, and Douglas Low. 1997. A taxonomy of obfuscating transformations.
- [12] Mai Hoang Dao, Thinh Hung Truong, and Dat Quoc Nguyen. 2021. Intent detection and slot filling for Vietnamese. *arXiv preprint arXiv:2104.02021* (2021).
- [13] Zakir Durumeric, Zane Ma, Drew Springall, Richard Barnes, Nick Sullivan, Elie Bursztein, Michael D Bailey, J Alex Halderman, and Vern Paxson. 2017. The Security Impact of HTTPS Interception.. In *NDSS*.
- [14] Fyrd. 2025. Can I use... Support tables for HTML5, CSS3, etc. <https://caniuse.com/?search=WebAssembly>. Accessed: 2025-05-24.
- [15] Ananda Mohon Ghosh and Katarina Grolinger. 2020. Edge-cloud computing for Internet of Things data analytics: Embedding intelligence in the edge with deep learning. *IEEE Transactions on Industrial Informatics* 17, 3 (2020), 2191–2200.
- [16] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. 2016. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *International conference on machine learning*. PMLR, 201–210.
- [17] Jayraj Gondaliya, Savani Savani, Vivek Sheetal Dhaduvali, and Gahangir Hossain. 2018. Hybrid security RSA algorithm in application of web service. In *2018 1st International Conference on Data Intelligence and Security (ICDIS)*. IEEE, 149–152.
- [18] Google XNNPACK Team. 2025. XNNPACK Test README. https://chromium.googlesource.com/external/github.com/google/XNNPACK/%2B/refs/heads/upstream/test_272573034/README.md. Accessed: 2025-05-24.
- [19] Maurice H Halstead. 1977. *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc.
- [20] Håkon Harnes and Donn Morrison. 2024. Cryptic Bytes: WebAssembly Obfuscation for Evading Cryptojacking Detection. *arXiv preprint arXiv:2403.15197* (2024).
- [21] Håkon Harnes and Donn Morrison. 2024. Cryptic Bytes: WebAssembly Obfuscation for Evading Cryptojacking Detection. *arXiv preprint arXiv:2403.15197* (2024).
- [22] Nima Shiri Harzevili, Jiho Shin, Junjie Wang, Song Wang, and Nachiappan Nagapan. 2023. Characterizing and understanding software security vulnerabilities in machine learning libraries. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 27–38.
- [23] Martin Heller. 2020. How to use TensorFlow in your browser. <https://www.infoworld.com/article/2259548/how-to-use-tensorflow-in-your-browser.html>. Accessed: 2025-05-24.
- [24] Ehsan Hesamifard, Hassan Takabi, and Mehdi Ghasemi. 2017. Cryptodl: Deep neural networks over encrypted data. *arXiv preprint arXiv:1711.05189* (2017).
- [25] inSpace. n.d.. *inSpace*. <https://inspace.chat/>. Accessed: 2025-09-11.
- [26] Sanjay Kariyappa and Moinuddin K Qureshi. 2020. Defending against model stealing attacks with adaptive misinformation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 770–778.
- [27] Hardik Kundraiya. 2025. 10 Top Node.js Libraries and Tools For Machine Learning. <https://www.corbado.com/blog/10-top-nodejs-libraries-machine-learning>. Accessed: 2025-05-24.
- [28] Zhongzheng Lai, Huaming Chen, Ruoxi Sun, Yu Zhang, Minhui Xue, and Dong Yuan. 2024. On Security Weaknesses and Vulnerabilities in Deep Learning Systems. *IEEE Transactions on Dependable and Secure Computing* (2024).
- [29] Taesung Lee, Benjamin Edwards, Ian Molloy, and Dong Su. 2019. Defending against neural network model stealing attacks using deceptive perturbations. In *2019 IEEE Security and Privacy Workshops (SPW)*. IEEE, 43–49.
- [30] Cullen Linn and Saumya Debray. 2003. Obfuscation of executable code to improve resistance to static disassembly. In *Proceedings of the 10th ACM conference on Computer and communications security*. 290–299.
- [31] Maybelline New York. n.d.. *Virtual Try-On Makeup Tools*. <https://www.maybelline.com/virtual-try-on-makeup-tools>. Accessed: 2025-09-11.
- [32] Thomas J McCabe. 1976. A complexity measure. *IEEE Transactions on software Engineering* 4 (1976), 308–320.
- [33] MDN Contributors. 2023. Compiling from Rust to WebAssembly. https://developer.mozilla.org/en-US/docs/WebAssembly/Rust_to_wasm. Accessed: 2025-05-24.
- [34] Microsoft. 2023. ONNX Runtime Web Documentation. <https://onnxruntime.ai/docs/api/js/>. Accessed: 2025-09-11.
- [35] Microsoft Open Source Team. 2021. ONNX Runtime Web: Running your machine learning model in the browser. <https://opensource.microsoft.com/blog/2021/09/02/onnx-runtime-web-running-your-machine-learning-model-in-browser/>. Accessed: 2025-05-24.
- [36] Smitha Milli, Ludwig Schmidt, Anca D Dragan, and Moritz Hardt. 2019. Model reconstruction from model explanations. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*. 1–9.
- [37] Trail of Bits. 2019. Manticore: A User-Friendly Symbolic Execution Framework for Binaries and Smart Contracts. <https://github.com/trailofbits/manticore>. Accessed: 2025-10-08.
- [38] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. 2019. Knockoff nets: Stealing functionality of black-box models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 4954–4963.
- [39] Stack Overflow. 2020. How to protect (obfuscate/DRM) trained model weights in TensorFlow.js? <https://stackoverflow.com/q/56919400>.
- [40] PKU-ASAL. 2025. SeeWasm: A Symbolic Execution Engine for WebAssembly Binaries. <https://github.com/PKU-ASAL/SeeWasm>. Accessed: 2025-10-08.
- [41] NF Prangan and J Wu. 2024. AI and computing horizons: cloud and edge in the modern era. *J Sens Actuator Netw* 13 (4): Article 44.
- [42] Adnan Siraj Rakin, Md Hafizul Islam Chowdhury, Fan Yao, and Deliang Fan. 2022. Deepsteal: Advanced model extractions leveraging efficient weight stealing in memories. In *2022 IEEE symposium on security and privacy (SP)*. IEEE, 1157–1174.
- [43] Daniel Smilkov, Nikhil Thorat, Yannick Assogba, Charles Nicholson, Nick Kreeger, Ping Yu, Shanqing Cai, Eric Nielsen, David Soegel, Stan Bileschi, et al. 2019. TensorFlow.js: Machine learning for the web and beyond. *Proceedings of Machine Learning and Systems* 1 (2019), 309–321.
- [44] Daniel Smilkov, Nikhil Thorat, Yannick Assogba, Charles Nicholson, Nick Kreeger, Ping Yu, Shanqing Cai, Eric Nielsen, David Soegel, Stan Bileschi, et al. 2019. TensorFlow.js: Machine learning for the web and beyond. *Proceedings of Machine Learning and Systems* 1 (2019), 309–321.
- [45] TinyGo Contributors. 2023. WebAssembly Guide. <https://tinygo.org/docs/guides/webassembly/>. Accessed: 2025-05-24.
- [46] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. Stealing machine learning models via prediction {APIs}. In *25th USENIX security symposium (USENIX Security)*. 601–618.
- [47] Florian Tramèr, Fan Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2016. Stealing machine learning models via prediction {APIs}. In *25th USENIX security symposium (USENIX Security)*. 601–618.
- [48] TVM Community. 2020. Compiling machine learning to WebAssembly and WebGPU. <https://tvm.apache.org/2020/05/14/compiling-machine-learning-to-webassembly-and-webgpu>. Accessed: 2025-05-24.
- [49] Nitesh Upadhyaya. 2024. Artificial intelligence in web development: Enhancing automation, personalization, and decision-making. *Artificial intelligence* 4, 1 (2024), 534–540.
- [50] Ronny Votel and Na Li. 2021. *Next-Generation Pose Detection with MoveNet and TensorFlow.js*. <https://blog.tensorflow.org/2021/05/next-generation-pose-detection-with-movenet-and-tensorflowjs.html>. Accessed: 2025-09-11.
- [51] WABT Contributors. 2023. wasm2wat tool documentation. <https://webassembly.github.io/wabt/doc/wasm2wat.1.html>. Accessed: 2025-05-24.
- [52] Qipeng Wang, Shiqi Jiang, Zhenpeng Chen, Xu Cao, Yuanchun Li, Aoyu Li, Yun Ma, Ting Cao, and Xuanzhe Liu. 2025. Anatomizing deep learning inference in web browsers. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–43.
- [53] Qipeng Wang, Shiqi Jiang, Zhenpeng Chen, Xu Cao, Yuanchun Li, Aoyu Li, Yun Ma, Ting Cao, and Xuanzhe Liu. 2025. Anatomizing deep learning inference in web browsers. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–43.
- [54] Zichao Wang, Angus Lamb, Evgeny Saveliev, Pashmina Cameron, Yordan Zaykov, José Miguel Hernández-Lobato, Richard E Turner, Richard G Baraniuk, Craig Barton, Simon Peyton Jones, et al. 2020. Instructions and guide for diagnostic questions: The neurips 2020 education challenge. *arXiv preprint arXiv:2007.12061* (2020).
- [55] Qixue Xiao, Kang Li, Deyue Zhang, and Weilin Xu. 2018. Security risks in deep learning implementations. In *2018 IEEE Security and privacy workshops (SPW)*. IEEE, 123–128.

- [56] Alon Zakai. 2011. Emscripten: an LLVM-to-JavaScript compiler. In *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications*. 301–312.
- [57] Jiliang Zhang, Shuang Peng, Yansong Gao, Zhi Zhang, and Qinghui Hong. 2023. APMSA: Adversarial perturbation against model stealing attacks. *IEEE Transactions on Information Forensics and Security* 18 (2023), 1667–1679.

Appendix

A Threat Model

The target of the attack in section 2 is the machine learning model deployed in JavaScript-based frontend frameworks and executed within the browser. The adversary aims to steal both the complete model architecture and its trained weights in order to fully reproduce a functionally equivalent model on the client side. We assume a strong adversary operating in an untrusted client environment, with the ability to observe, analyze, and manipulate client-side execution, as well as to monitor network interactions between the browser and the server. Under this threat model, the adversary may intercept model-related network transmissions, access browser caches and local storage, inspect loaded JavaScript or WebAssembly binaries, and apply customized analysis or reverse-engineering tools to reconstruct the original model. Unlike server-side deployments, frontend models are inherently exposed to such inspection, making them vulnerable to intellectual property leakage. The proposed method is designed to protect the confidentiality and integrity of frontend-deployed models by preventing effective extraction, reverse engineering, and unauthorized reuse. Even when the adversary has access to the runtime environment and execution artifacts, our approach aims to ensure that critical model information cannot be reliably recovered or reconstructed.

B WAMO Design Principles

This system aims to compile models of any format into a more secure Wasm format, enabling front-end inference by invoking Wasm models instead of traditional frameworks like TensorFlow.js. Its design focuses on secure model deployment on the front end, addressing vulnerabilities such as easy theft of model parameters from browser caches and reverse engineering of model logic. The architecture follows these principles:

- (1) The system is designed with model protection in mind during conversion and deployment, making model parameters hard to access. It packages both the model architecture and weights along with the runtime, eliminating the need for external model files or operator libraries during execution.
- (2) The system supports major training frameworks (TensorFlow, PyTorch, ONNX), unifying their models into a Wasm format to ensure cross-platform deployability.
- (3) The system enforces protection on the model, ensuring that the resulting Wasm model file resists static binary analysis. Even if an attacker acquires the Wasm model file locally in the browser, conventional binary analysis tools are unlikely to reveal the model's internal structure.
- (4) To enable practical use, the system automates model reading, conversion, and security enhancement, allowing direct deployment of secured models on the Web.

C Our Operator Library

The following figure 10 complements Section 4.1. It shows several types of models supported by our self-developed Wasm-friendly operator library. For each model category, we list some commonly implemented operators, with the total number exceeding 140.

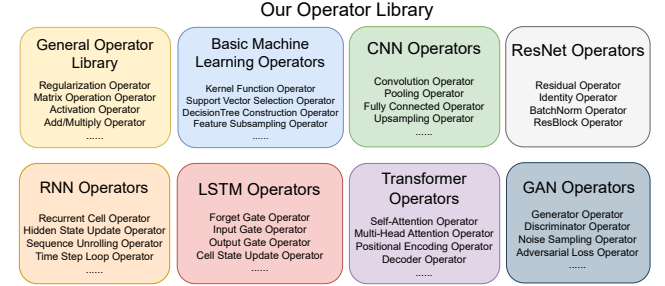


Figure 10: Operators in our custom library.

D Obfuscation Algorithms and Effects

This section presents several supplementary obfuscation algorithms from Section 5, along with comparative figures illustrating the effects before and after obfuscation.

The following algorithm 1 is the pseudocode of the concrete implementation process for operator execution path perturbation in Section 5.2, showing how execution path perturbation is achieved by invoking several key functions. Figure 11 shows the final obfuscation effect of operator execution path perturbation.

Algorithm 1 Operator Execution Path Perturbation

Require: Instruction sequence $\Phi = [I_1, \dots, I_n]$

Ensure: Obfuscated function F' , basic block set B , state mapping S

```

1:  $B \leftarrow \emptyset, S \leftarrow \emptyset$ 
2:  $b_{\text{start}} \leftarrow 1$ 
3: for  $i = 1$  to  $n$  do
4:   if  $I_i$  is control instruction then
5:      $b \leftarrow [b_{\text{start}}, i]$ 
6:     Record stack delta:  $\Delta_s(b)$ 
7:     Append  $b$  to  $B$ 
8:      $b_{\text{start}} \leftarrow i + 1$ 
9:   end if
10: end for
11: for each block  $b \in B$  do
12:    $S[b] \leftarrow$  unique state ID
13: end for
14: Shuffle  $B$ 
15: for each block  $b \in B$  do
16:    $\text{Succ}(b) \leftarrow$  set of successor blocks
17:   Set state updates for  $b$ :  $s_b \rightarrow s_{\text{next}}$ 
18: end for
19: Build  $\text{br\_table}$  mapping  $S[b] \mapsto b$ 
20:  $F' \leftarrow$  initialize state; loop:  $\text{br\_table}$  dispatch; update state
21: return ( $F', B, S$ )

```

Figure 12 illustrates the obfuscation effect of operator dispatch obfuscation from Section 5.3. During implementation, we add the target function indices of call instructions to the elem table, which

Algorithm 2 Operator Dispatch Obfuscation

Require: Instruction sequence $\Phi = [I_1, \dots, I_n]$, function index map $\mathcal{F}$, existing elem table T

Ensure: Updated elem table T , obfuscated call set C^o

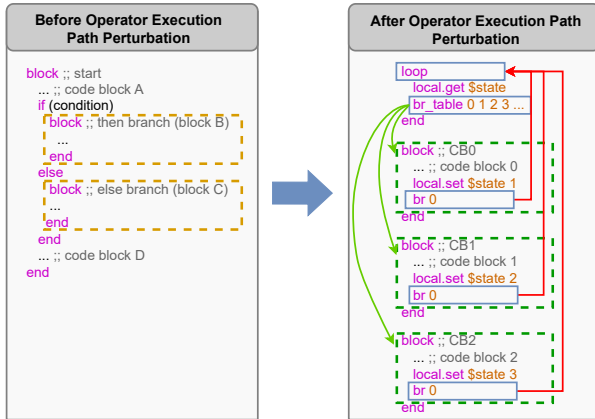
```

1: Initialize  $C^o \leftarrow \emptyset$ 
2: for each instruction  $I_i \in \Phi$  do
3:   if  $I_i$  is call targeting function  $f$  then
4:      $idx_f \leftarrow \mathcal{F}(f)$ 
5:     if  $idx_f \notin T$  then
6:       Append  $idx_f$  to  $T$ 
7:     end if
8:      $pos \leftarrow \text{posOf}(idx_f, T)$ 
9:     Replace  $I_i$  with i32.const  $pos$ 
10:    Append call_indirect(type  $t_f$ )
11:    Add  $I_i$  to  $C^o$ 
12:   end if
13: end for
14: return  $(T, C^o)$ 

```

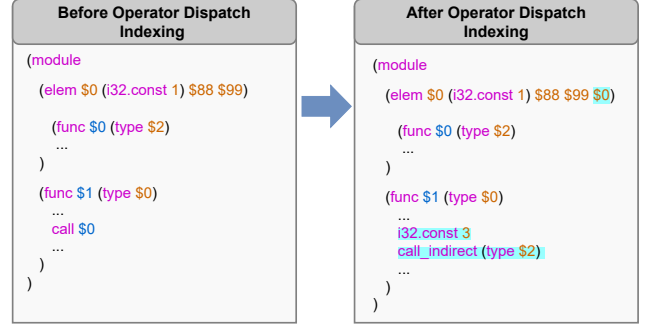
Table 5: Local system configuration

Parameter	Embedded Workloads
CPU Model	Ryzen AI 9 HX 370 (12C/24T)
RAM	32 GB DDR5, 111,186 MB/s
SSD	WD SN560 NVMe (4584/1629 MB/s)
GPU	RTX 4060 Laptop, 8 GB, 187,182 MB/s
Cache	L1: 49 KB/core, L3: 16 MB
OS	Windows 11 (Build 26100)
GPU Driver	Ver. 32.0.15.6607 (2024/10/20)
Threads	18.7× single-thread throughput
VM Usage	Max 34,554 MB, used 29,588 MB
DirectX	DirectX 12
Compiler	emcc 3.1.68 (Emscripten, clang 20.0.0git)

**Figure 11: Control flow reconstruction in operator execution path perturbation.**

defines the set of functions that can be jumped to by index. The original call instruction is replaced with an `i32.const` plus `call_indirect` sequence. For example, if a function f has a type index of 2 and is

at position 3 in the elem table, the obfuscated instruction sequence becomes: `i32.const 3, call_indirect (type 2)`.

**Figure 12: The effect of operator dispatch obfuscation**

Algorithm 2 illustrates the concrete implementation process of operator execution path perturbation.

E Experimental Environment Settings

This section provides supplementary explanations for Section 6.1. Table 5 presents the specific parameters of our experimental setup as well as the compiler settings. The tested models include two simple machine learning models (one Random Forest and one SVM), two GNNs, two CNNs, one ResNet, two standard RNNs, two LSTMs, two Transformer models, and one GAN.

F Equations

This section presents the formulas and metric definitions involved in the evaluation (Section 6) along with detailed explanations of each formula. Cyclomatic complexity and halstead complexity quantifies are used to measure the model's resistance to static analysis before and after obfuscation (Section 6.2). Memory distribution normalized entropy is used to assess the model's resistance to dynamic analysis (Section 6.3).

Cyclomatic complexity is used to measure the complexity of a program's control flow. It is computed based on the control flow graph (CFG) and reflects the number of branches and the complexity of execution paths. The more complex a function, the higher its cyclomatic complexity. For a function f , its cyclomatic complexity M can be calculated by Equation (1), where E , N , and P denote the number of edges, nodes, and connected components in the CFG of function, respectively. Using the Wasp tool, we generated the CFG for each function in the Wasm model file and the overall function call graph. To evaluate the cyclomatic complexity of the entire program, we summed the product of each function's cyclomatic complexity and its runtime call count C_i , as shown in Equation(2), where C_i represents the number of times the function is invoked at runtime.

$$M = E - N + 2P \quad (1)$$

$$M_{\text{total}} = \sum_{i=1}^n M_i \cdot C_i \quad (2)$$

Halstead complexity quantifies the complexity of a program based on the counts of operators and operands, reflecting code

maintainability and reverse-engineering difficulty. Let n_1 denote the number of distinct operators in the program (including procedure names, delimiters, and operator symbols), and n_2 the number of distinct operands. Let N_1 and N_2 represent the total occurrences of all operators and operands, respectively. The complexity metrics (effectiveness criteria) include program volume, difficulty, program length, vocabulary size, effort, and estimated number of errors.

$$\text{Vocabulary } (n) = n_1 + n_2 \quad (3)$$

$$\text{Program length } (N) = N_1 + N_2 \quad (4)$$

$$\text{Volume } (V) = N \cdot \log_2 n \quad (5)$$

$$\text{Difficulty } (D) = \frac{n_1}{2} \cdot \frac{N_2}{n_2} \quad (6)$$

$$\text{Effort } (E) = D \cdot V \quad (7)$$

Entropy quantifies the randomness of memory accesses during inference, as defined in Equations (8) and (9). Higher values indicate more dispersed and less predictable patterns, enhancing resistance to dynamic analysis. Here, p_i is the probability that the i -th memory block is accessed, computed as the ratio of its accesses to the total; k denotes the number of memory blocks; and u is the number of unique addresses accessed. Normalization by $\log_2 u$ provides a scale-independent metric, accurately reflecting memory access complexity and randomness.

$$H = - \sum_{i=1}^k p_i \log_2 p_i \quad (8)$$

$$H_{\text{normalized}} = \frac{H}{\log_2 u} = - \frac{\sum_{i=1}^k p_i \log_2 p_i}{\log_2 u} \quad (9)$$

G Related Work

G.1 Model Attacks. Existing research on model extraction mainly falls into four categories. Query-based cloning trains substitute models by sending inputs and collecting outputs; probability distributions allow boundary reconstruction [47], while even label-only outputs enable transferable cloning [38], with defenses via output perturbation [26]. Side-channel attacks [42] exploit runtime

memory or cache behaviors to extract parameters. Interpretability-assisted methods [36] use gradients, saliency maps, or activations to infer structures. Synthetic sample-driven attacks bypass queries, e.g., using evolutionary strategies [12] to generate data and combining membership inference to expand distribution coverage [54]. Unlike these, our attack leverages browser deployment: intercepting traffic or retrieving browser-cached Wasm and weight files allows full model reconstruction [39].

G.2 Model Conversion. Currently, two main approaches exist for converting traditional models to WebAssembly. The first downloads model formats (e.g., ONNX, TFLite) as static files [35], which are then interpreted in the browser by JavaScript or Wasm parsers. The second implements model parsing, graph execution, and operators in a high-level language, compiled into Wasm modules for browser execution, as in TVM.js [48]. In both cases, model structure and weights must be separately loaded, leaving the runtime exposed at the frontend. Some work also attempts automatic code generation, translating models into C functions or computation expressions and compiling them into Wasm for static deployment, such as XNNPack [18]. However, these tools usually support only operator-level conversion, whereas our system achieves full transformation from traditional models to C++, ensuring compatibility after compilation into Wasm.

G.3 Wasm Obfuscation. Currently, three main obfuscation approaches exist to enhance the security of Wasm model files, each targeting different intermediate forms of the model. The first is source-level obfuscation, such as using Tigress [1] to apply control-flow flattening or dummy code insertion to C/C++ source code. This requires models to be restored into compilable C/C++ form [20], offering strong obfuscation but producing large binaries. The second is IR-level obfuscation [7], which perturbs WebAssembly modules before loading through random replacements, redundant instructions, or local control-flow disruptions. This provides faster protection of model logic but may compromise executability or performance. The third is bytecode-level obfuscation [7], customized to Wasm's structured memory model, leveraging its storage characteristics to maximize correctness and efficiency while preserving model logic. Existing Wasm obfuscation tools, however, mainly target the MVP semantic set and rarely address advanced features such as SIMD.